

Лекция 19. Паттерн репозиторий и контейнер внедрения зависимости

Примените лучшие архитектурные практики к приложению и используйте Coil для загрузки и отображения изображений.

Введение

В предыдущем уроке вы узнали, как получить данные из веб-сервиса, попросив ViewModel получить URL-адреса фотографий Марса из сети с помощью API-сервиса. Хотя этот подход работает и прост в реализации, он не очень хорошо масштабируется по мере роста вашего приложения и необходимости работы с несколькими источниками данных. Чтобы решить эту проблему, лучшие практики архитектуры Android рекомендуют разделять слой пользовательского интерфейса и слой данных.

В этом уроке вы рефакторите приложение **Mars Photos** на отдельные слои пользовательского интерфейса и данных. Вы узнаете, как реализовать паттерн **репозитория** и использовать **инъекцию зависимостей**. Инъекция зависимостей создает более гибкую структуру кодирования, которая помогает при разработке и тестировании.

Необходимые условия

- Уметь получать JSON из веб-сервиса REST и разбирать эти данные на объекты Kotlin с помощью библиотек Retrofit и Serialization (kotlinx.serialization).
- Знание того, как использовать веб-сервис REST.
- Уметь реализовывать корутины в своем приложении.

Что вы узнаете

- Паттерн репозитория
- Инъекция зависимостей

Что вы создадите

- Измените приложение **Mars Photos**, чтобы разделить его на слой пользовательского интерфейса и слой данных.
- При выделении слоя данных вы будете реализовывать паттерн репозитория.
- Используйте инъекцию зависимостей для создания слабосвязанной кодовой базы.

Что вам нужно

- Компьютер с современным веб-браузером, например последней версией Chrome.
- Получите стартовый код

Чтобы начать работу, загрузите стартовый код:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-mars-photos.git
```

```
$ cd basic-android-kotlin-compose-training-mars-photos  
$ git checkout repo-starter
```

Разделение слоя пользовательского интерфейса и слоя данных

Зачем нужны разные слои?

Разделение кода на разные слои делает ваше приложение более масштабируемым, надежным и легким для тестирования. Наличие нескольких слоев с четко определенными границами также облегчает работу нескольких разработчиков над одним и тем же приложением без негативного влияния друг на друга.

Рекомендуемая архитектура приложений Android гласит, что приложение должно иметь как минимум слой пользовательского интерфейса и слой данных.

В этом уроке вы сосредоточитесь на слое данных и внесете изменения, чтобы ваше приложение соответствовало рекомендуемым лучшим практикам.

Что такое слой данных?

Слой данных отвечает за бизнес-логику вашего приложения, а также за поиск и сохранение данных для вашего приложения. Слой данных предоставляет данные слою пользовательского интерфейса, используя шаблон однонаправленного потока данных. Данные могут поступать из разных источников, например, из сетевого запроса, локальной базы данных или файла на устройстве.

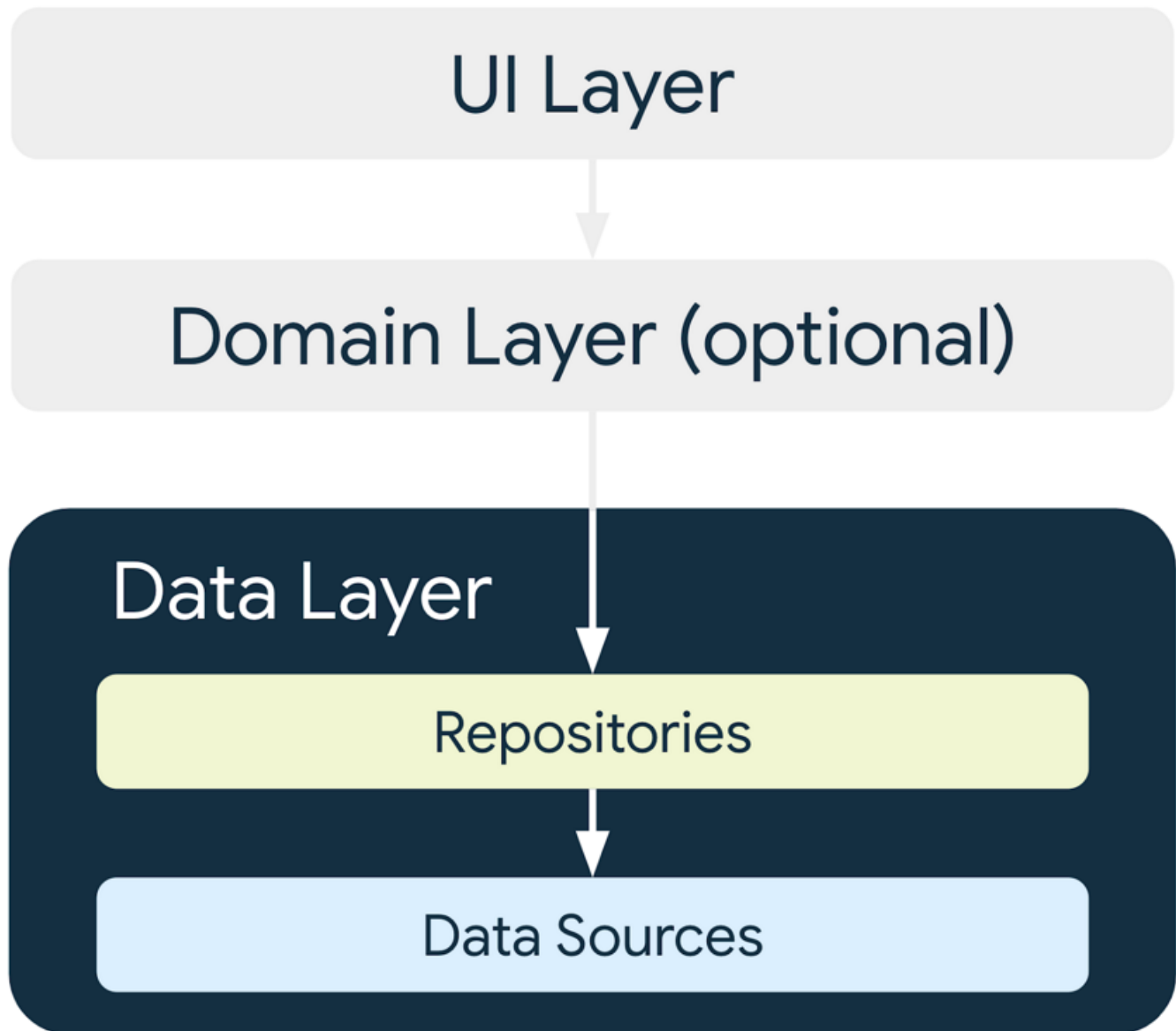
У приложения может быть даже несколько источников данных. Когда приложение открывается, оно получает данные из локальной базы данных на устройстве, что является первым источником. Во время работы приложения оно делает сетевой запрос ко второму источнику, чтобы получить новые данные.

Благодаря тому, что данные находятся в отдельном слое от кода пользовательского интерфейса, вы можете вносить изменения в одну часть кода, не затрагивая другую. Такой подход является частью принципа проектирования, называемого разделением обязанностей. Часть кода фокусируется на своей собственной проблеме и инкапсулирует ее внутреннюю работу от другого кода. **Инкапсуляция** - это форма сокрытия внутренней работы кода от других разделов кода. Когда одному разделу кода нужно взаимодействовать с другим разделом кода, он делает это через интерфейс.

Задача слоя пользовательского интерфейса - отображать предоставленные ему данные.

Пользовательский интерфейс больше не извлекает данные, поскольку этим занимается слой данных.

Слой данных состоит из одного или нескольких репозиториях. Сами репозитории содержат ноль или более источников данных.



Лучшие практики требуют, чтобы приложение имело репозиторий для каждого типа источников данных, которые использует ваше приложение.

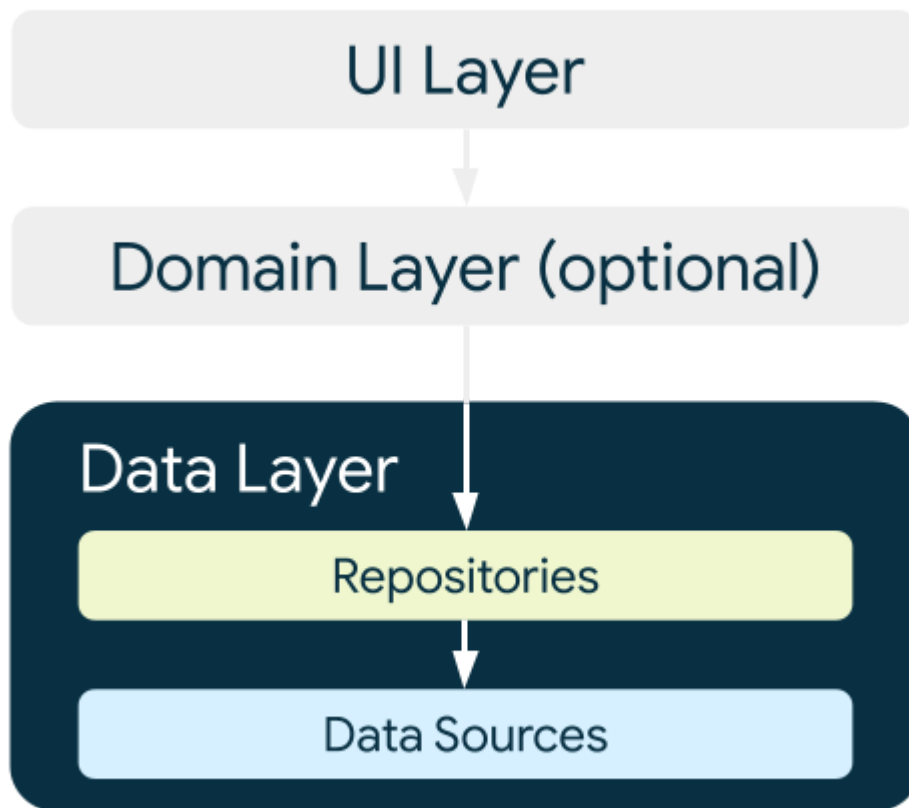
В этом уроке у приложения один источник данных, поэтому после рефакторинга кода у него будет одно хранилище. В этом приложении репозиторий, получающий данные из интернета, выполняет обязанности источника данных. Для этого он делает сетевой запрос к API. Если кодирование источника данных более сложное или добавлены дополнительные источники данных, обязанности источника данных инкапсулируются в отдельные классы источников данных, а репозиторий отвечает за управление всеми источниками данных.

Что такое репозиторий?

В общем случае класс репозитория:

- Предоставляет данные остальным частям приложения.
- Централизует изменения данных.
- Разрешает конфликты между несколькими источниками данных.
- Абстрагирует источники данных от остальной части приложения.
- Содержит бизнес-логику.

Приложение **Mars Photos** имеет единственный источник данных, которым является вызов сетевого API. В нем нет никакой бизнес-логики, поскольку оно просто получает данные. Данные предоставляются приложению через класс репозитория, который абстрагируется от источника данных.



{style=«width:400px»}

Создание слоя данных

Сначала необходимо создать класс репозитория. В руководстве для разработчиков Android говорится, что классы хранилищ называются в честь данных, за которые они отвечают. Соглашение об именовании репозитория выглядит так: тип данных + репозиторий. В вашем приложении это **MarsPhotosRepository**.

Создайте репозиторий

- Щелкните правой кнопкой мыши на `com.example.marsphotos` и выберите **New > Package**.
- В диалоговом окне введите данные.
- Щелкните правой кнопкой мыши на пакете данных и выберите **New > Kotlin Class/File**.
- В диалоговом окне выберите **Interface** и введите `MarsPhotosRepository` в качестве имени интерфейса.
- Внутри интерфейса `MarsPhotosRepository` добавьте абстрактную функцию `getMarsPhotos()`, которая возвращает список объектов `MarsPhoto`. Она вызывается из корутины, поэтому объявите ее с помощью `suspend`.

```
import com.example.marsphotos.model.MarsPhoto

interface MarsPhotosRepository {
    suspend fun getMarsPhotos(): List<MarsPhoto>
}
```

- Ниже объявления интерфейса создайте класс `NetworkMarsPhotosRepository` для реализации интерфейса `MarsPhotosRepository`.
- Добавьте интерфейс `MarsPhotosRepository` к объявлению класса. Поскольку вы не переопределили абстрактный метод интерфейса, появится сообщение об ошибке. На следующем шаге мы устраним эту ошибку.

```
interface MarsPhotosRepository {
    /** Fetches list of MarsPhoto from marsApi */
    suspend fun getMarsPhotos(): List<MarsPhoto>
}

class NetworkMarsPhotosRepository() : MarsPhotosRepository {
```

- Внутри класса `NetworkMarsPhotosRepository` переопределите абстрактную функцию `getMarsPhotos()`. Эта функция возвращает данные, полученные в результате вызова `MarsApi.retrofitService.getPhotos()`.

```
import com.example.marsphotos.network.MarsApi

class NetworkMarsPhotosRepository() : MarsPhotosRepository {
    override suspend fun getMarsPhotos(): List<MarsPhoto> {
        return MarsApi.retrofitService.getPhotos()
    }
}
```

Далее необходимо обновить код `ViewModel`, чтобы он использовал репозиторий для получения данных, как это советуют лучшие практики Android.

- Откройте файл `ui/screens/MarsViewModel.kt`.
- Прокрутите вниз до метода `getMarsPhotos()`.
- Замените строку `«val listResult = MarsApi.retrofitService.getPhotos()»` следующим кодом:

```
import com.example.marsphotos.data.NetworkMarsPhotosRepository

val marsPhotosRepository = NetworkMarsPhotosRepository()
val listResult = marsPhotosRepository.getMarsPhotos()
```

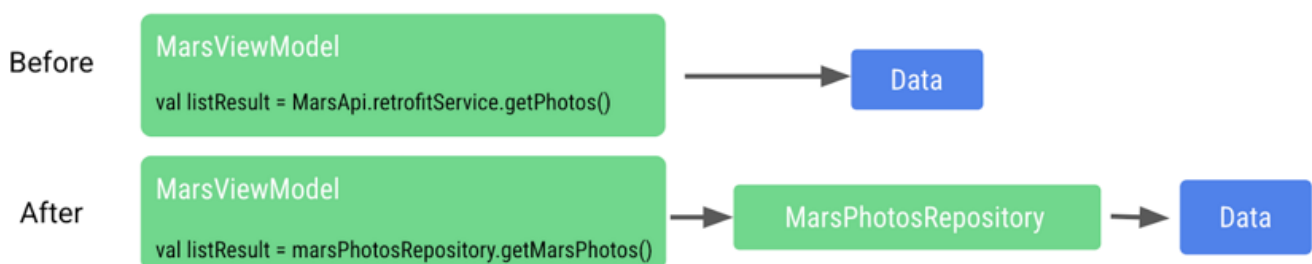
```

private fun getMarsPhotos() {
    viewModelScope.launch { this: CoroutineScope
        marsUiState = try {
            val marsPhotosRepository = NetworkMarsPhotosRepository()
            val listResult = marsPhotosRepository.getMarsPhotos()
            MarsUiState.Success( photos: "Success. ${listResult.size} Mars photos retrieved")
        } catch (e: IOException) {
            MarsUiState.Error
        } catch (e: HttpException) {
            MarsUiState.Error
        }
    }
}

```

- Запустите приложение. Обратите внимание, что отображаемые результаты такие же, как и предыдущие.

Вместо того чтобы ViewModel напрямую выполняла сетевой запрос на получение данных, их предоставляет хранилище. Модель ViewModel больше не обращается напрямую к коду MarsApi.



Такой подход помогает сделать код, получающий данные, слабо связанным с ViewModel. Свободная связь позволяет вносить изменения во ViewModel или хранилище без негативного влияния на другое, пока в хранилище есть функция getMarsPhotos().

Теперь мы можем вносить изменения в реализацию внутри хранилища, не затрагивая вызывающую функцию. В больших приложениях это изменение может поддерживать несколько вызывающих пользователей.

Инъекция зависимостей

Часто для работы классов требуются объекты других классов. Когда класс требует другой класс, требуемый класс называется **зависимостью**.

В следующих примерах объект Car зависит от объекта Engine.

Существует два способа получения классом этих необходимых объектов. Один из них заключается в том, что класс сам инстанцирует требуемый объект.

```

interface Engine {
    fun start()
}

```

```
class GasEngine : Engine {
    override fun start() {
        println("GasEngine started!")
    }
}

class Car {

    private val engine = GasEngine()

    fun start() {
        engine.start()
    }
}

fun main() {
    val car = Car()
    car.start()
}
```

Другой способ - передать требуемый объект в качестве аргумента.

```
interface Engine {
    fun start()
}

class GasEngine : Engine {
    override fun start() {
        println("GasEngine started!")
    }
}

class Car(private val engine: Engine) {
    fun start() {
        engine.start()
    }
}

fun main() {
    val engine = GasEngine()
    val car = Car(engine)
    car.start()
}
```

Заставить класс инстанцировать нужные объекты легко, но такой подход делает код негибким и более сложным для тестирования, поскольку класс и нужный объект тесно связаны между собой.

Вызывающий класс должен вызвать конструктор объекта, что является деталью реализации. Если конструктор меняется, то и вызывающий код должен измениться.

Чтобы сделать код более гибким и адаптируемым, класс не должен инстанцировать объекты, от которых он зависит. Объекты, от которых он зависит, должны инстанцироваться вне класса, а затем передаваться в него. Такой подход позволяет создать более гибкий код, поскольку класс больше не привязан жестко к одному конкретному объекту. Реализация требуемого объекта может измениться без необходимости модифицировать вызывающий код.

Продолжая предыдущий пример, если нужен `ElectricEngine`, его можно создать и передать в класс `Car`. При этом класс `Car` не нужно никак модифицировать.

```
interface Engine {
    fun start()
}

class ElectricEngine : Engine {
    override fun start() {
        println("ElectricEngine started!")
    }
}

class Car(private val engine: Engine) {
    fun start() {
        engine.start()
    }
}

fun main() {
    val engine = ElectricEngine()
    val car = Car(engine)
    car.start()
}
```

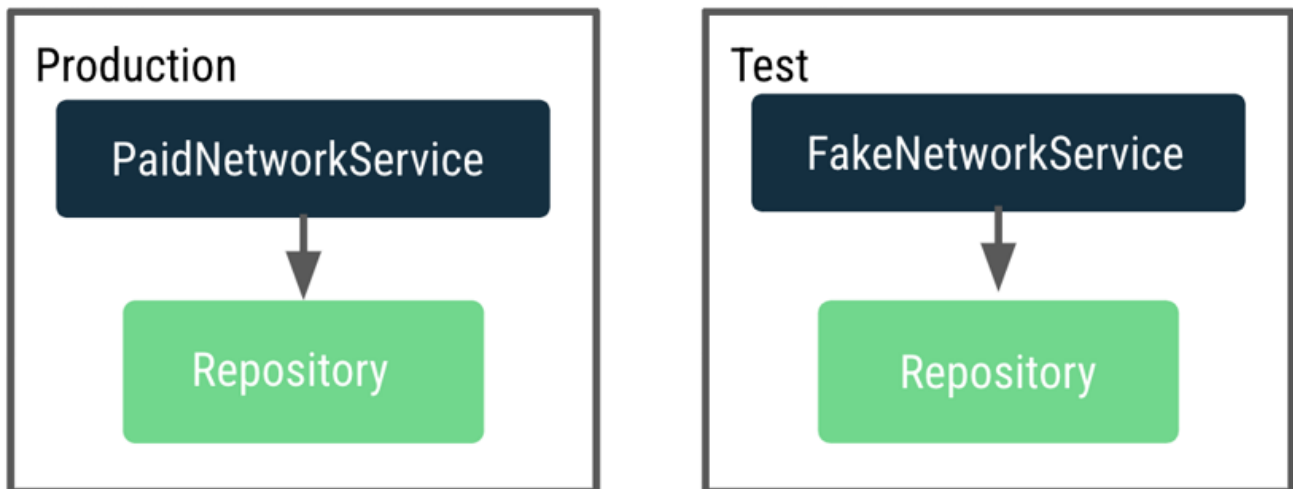
Передача необходимых объектов называется инъекцией зависимостей (DI). Она также известна как инверсия управления.

DI - это когда зависимость предоставляется во время выполнения вместо того, чтобы быть жестко закодированной в вызывающем классе.

Реализация инъекции зависимостей:

- Помогает повторно использовать код. Код не зависит от конкретного объекта, что обеспечивает большую гибкость.
- Облегчает рефакторинг. Код слабо связан, поэтому рефакторинг одного участка кода не влияет на другой участок кода.
- Помогает при тестировании. Тестовые объекты можно передавать во время тестирования.
- Одним из примеров того, как DI может помочь в тестировании, является тестирование кода вызова сети. В этом тесте вы пытаетесь проверить, что сетевой вызов выполнен и что данные возвращены. Если бы вам пришлось платить каждый раз, когда вы делаете сетевой запрос во время тестирования, вы могли бы решить пропустить тестирование этого кода, так как это может быть дорого. А теперь представьте, что мы можем подделать сетевой запрос для тестирования.

Насколько счастливее (и богаче) вы от этого станете? Для тестирования вы можете передать в хранилище тестовый объект, который при вызове возвращает фальшивые данные, не выполняя реального сетевого вызова.



Мы хотим сделать ViewModel тестируемой, но в настоящее время она зависит от репозитория, который выполняет реальные сетевые вызовы. При тестировании с реальным производственным репозиторием он делает много сетевых вызовов. Чтобы решить эту проблему, вместо того чтобы ViewModel создавала репозиторий, нам нужен способ динамически определять и передавать экземпляр репозитория для использования в производстве и тестировании.

Этот процесс осуществляется путем реализации **контейнера приложения**, который предоставляет репозиторий MarsViewModel.

Контейнер - это объект, который содержит зависимости, необходимые приложению. Эти зависимости используются во всем приложении, поэтому они должны находиться в общем месте, которое могут использовать все действия. Вы можете создать подкласс класса `Application` и хранить в нем ссылку на контейнер.

Создание контейнера приложения

- Щелкните правой кнопкой мыши на пакете данных и выберите New > Kotlin Class/File.
- В диалоговом окне выберите Interface и введите AppContainer в качестве имени интерфейса.
- Внутри интерфейса AppContainer добавьте абстрактное свойство marsPhotosRepository типа MarsPhotosRepository.

```
interface AppContainer {
    val marsPhotosRepository: MarsPhotosRepository
}
```

Ниже определения интерфейса создайте класс `DefaultAppContainer`, который реализует интерфейс AppContainer.

Из файла `network/MarsApiService.kt` переместите код переменных `BASE_URL`, `retrofit` и `retrofitService` в класс `DefaultAppContainer`, чтобы все они находились в контейнере, поддерживающем зависимости.

```
import retrofit2.Retrofit
import com.example.marsphotos.network.MarsApiService
import
com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
import kotlinx.serialization.json.Json
import okhttp3.MediaType.Companion.toMediaType

class DefaultAppContainer : AppContainer {

    private const val BASE_URL = 'https://android-kotlin-fun-mars-
server.appspot.com'

    private val retrofit: Retrofit = Retrofit.Builder()

.addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
    .baseUrl(BASE_URL)
    .build()

    private val retrofitService: MarsApiService by lazy {
        retrofit.create(MarsApiService::class.java)
    }
}
```

- Для переменной `BASE_URL` удалите ключевое слово `const`. Удаление `const` необходимо, потому что `BASE_URL` больше не является переменной верхнего уровня и теперь является свойством класса `DefaultAppContainer`. Переделайте его на верблужий регистр `baseUrl`.
- Для переменной `retrofitService` добавьте модификатор видимости `private`. Модификатор `private` добавлен потому, что переменная `retrofitService` используется только внутри класса свойством `marsPhotosRepository`, поэтому она не должна быть доступна вне класса. Класс `DefaultAppContainer` реализует интерфейс `AppContainer`, поэтому нам нужно переопределить свойство `marsPhotosRepository`. После переменной `retrofitService` добавьте следующий код:

```
override val marsPhotosRepository: MarsPhotosRepository by lazy {
    NetworkMarsPhotosRepository(retrofitService)
}
```

Завершенный класс `DefaultAppContainer` должен выглядеть следующим образом:

```
class DefaultAppContainer : AppContainer {

    private val baseUrl = 'https://android-kotlin-fun-mars-server.appspot.com'
```

```
//Use the Retrofit builder to build a retrofit object using a
kotlinx.serialization converter

private val retrofit = Retrofit.Builder()

.addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
    .baseUrl(baseUrl)
    .build()

private val retrofitService: MarsApiService by lazy {
    retrofit.create(MarsApiService::class.java)
}

override val marsPhotosRepository: MarsPhotosRepository by lazy {
    NetworkMarsPhotosRepository(retrofitService)
}
}
```

- Откройте файл `data/MarsPhotosRepository.kt`. Теперь мы передаем `retrofitService` в `NetworkMarsPhotosRepository`, и вам необходимо изменить класс `NetworkMarsPhotosRepository`. В объявлении класса `NetworkMarsPhotosRepository` добавьте параметр конструктора `marsApiService`, как показано в следующем коде.

```
import com.example.marsphotos.network.MarsApiService

class NetworkMarsPhotosRepository(
    private val marsApiService: MarsApiService
) : MarsPhotosRepository {
```

- В классе `NetworkMarsPhotosRepository` в функции `getMarsPhotos()` измените оператор возврата, чтобы получать данные из `marsApiService`.

```
override suspend fun getMarsPhotos(): List<MarsPhoto> = marsApiService.getPhotos()
}
```

- Удалите следующий импорт из файла `MarsPhotosRepository.kt`.

```
// Remove
import com.example.marsphotos.network.MarsApi
```

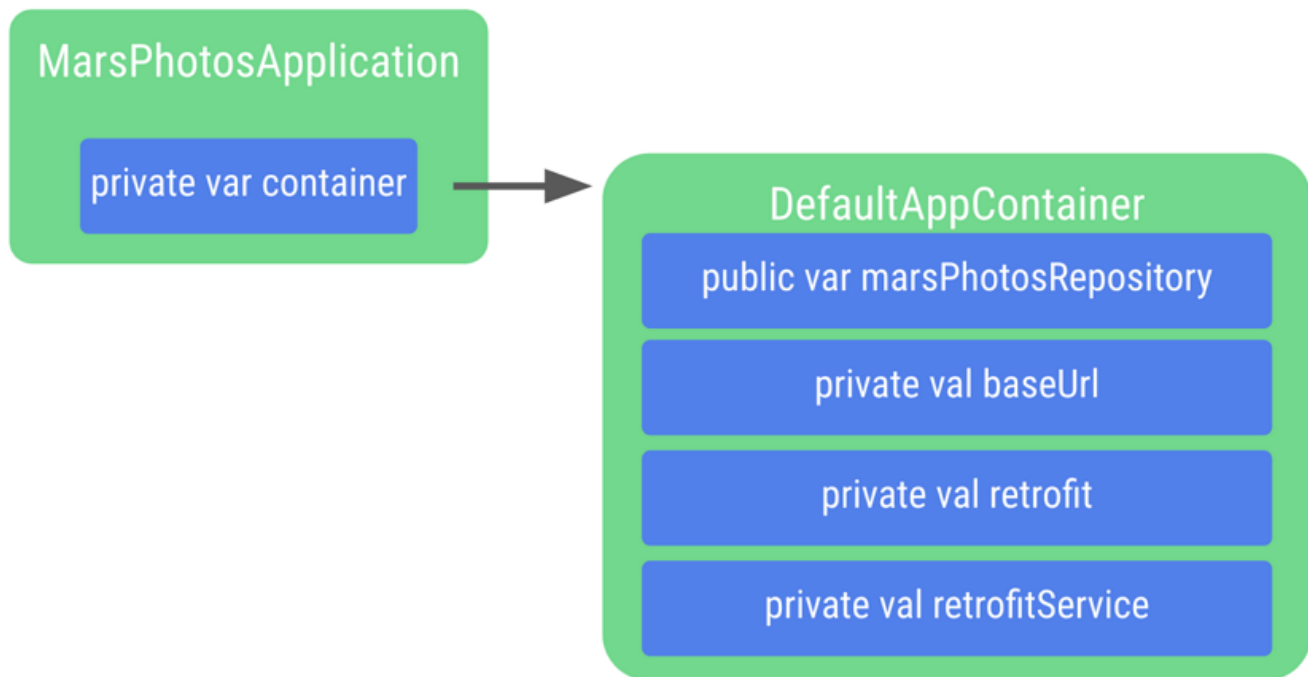
Из файла `network/MarsApiService.kt` мы удалили весь код из объекта. Теперь мы можем удалить оставшееся объявление объекта, поскольку оно больше не нужно.

- Удалите следующий код:

```
object MarsApi {  
  
}
```

Присоединение контейнера приложения к приложению

Шаги в этом разделе соединяют объект приложения с контейнером приложения, как показано на следующем рисунке.



- Щелкните правой кнопкой мыши на `com.example.marsphotos` и выберите New > Kotlin Class/File.
-

В диалоговом окне введите `MarsPhotosApplication`. Этот класс наследуется от объекта приложения, поэтому вам нужно добавить его в объявление класса.

```
import android.app.Application  
  
class MarsPhotosApplication : Application() {  
}
```

- Внутри класса `MarsPhotosApplication` объявите переменную `container` типа `AppContainer` для хранения объекта `DefaultAppContainer`. Переменная инициализируется во время вызова функции `onCreate()`, поэтому ее необходимо пометить модификатором `lateinit`.

```
import com.example.marsphotos.data.AppContainer  
import com.example.marsphotos.data.DefaultAppContainer
```

```
lateinit var container: AppContainer
override fun onCreate() {
    super.onCreate()
    container = DefaultAppContainer()
}
```

- Полный файл `MarsPhotosApplication.kt` должен выглядеть следующим образом:

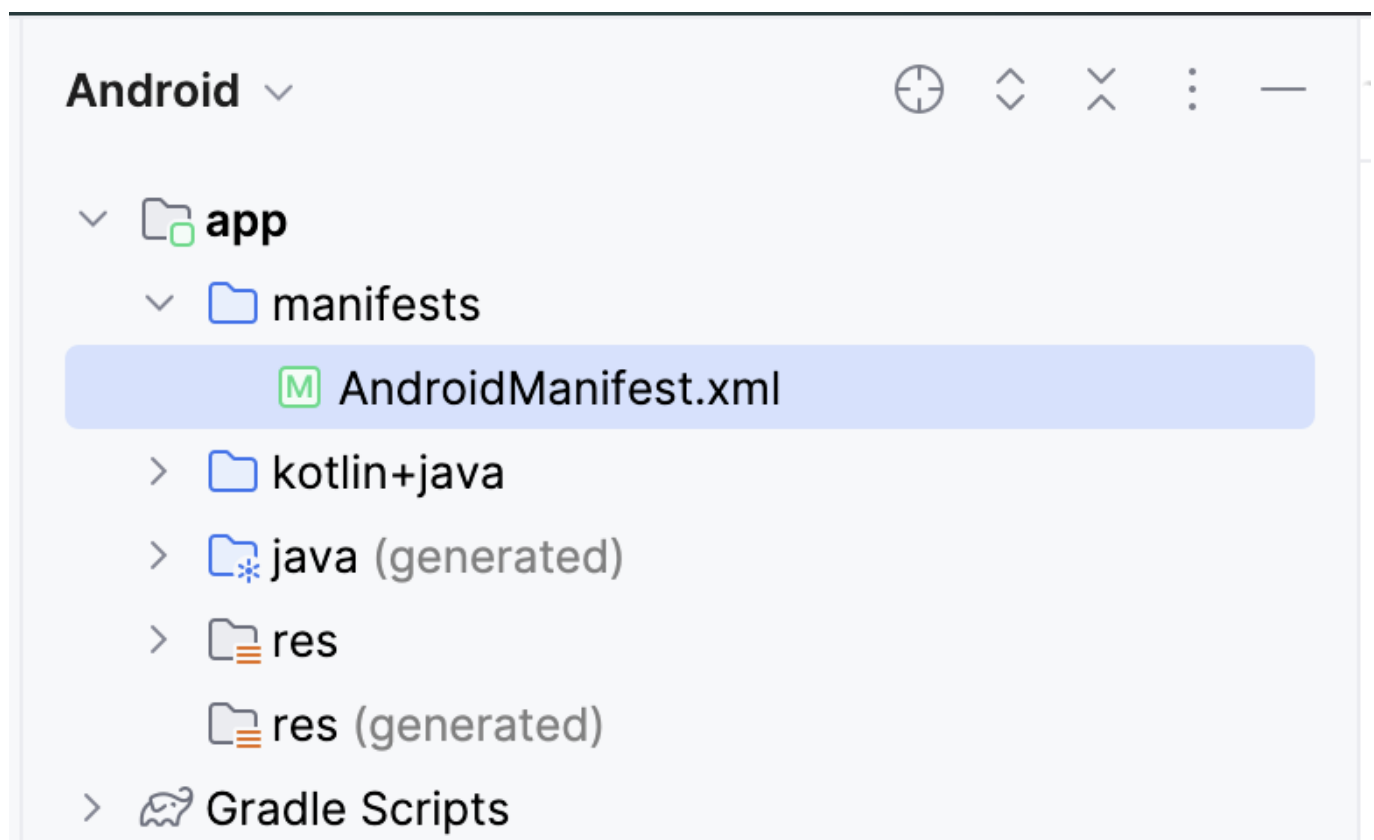
```
package com.example.marsphotos

import android.app.Application
import com.example.marsphotos.data.AppContainer
import com.example.marsphotos.data.DefaultAppContainer

class MarsPhotosApplication : Application() {
    lateinit var container: AppContainer
    override fun onCreate() {
        super.onCreate()
        container = DefaultAppContainer()
    }
}
```

Вам нужно обновить манифест Android, чтобы приложение использовало класс приложения, который вы только что определили.

- Откройте файл `manifests/AndroidManifest.xml`.

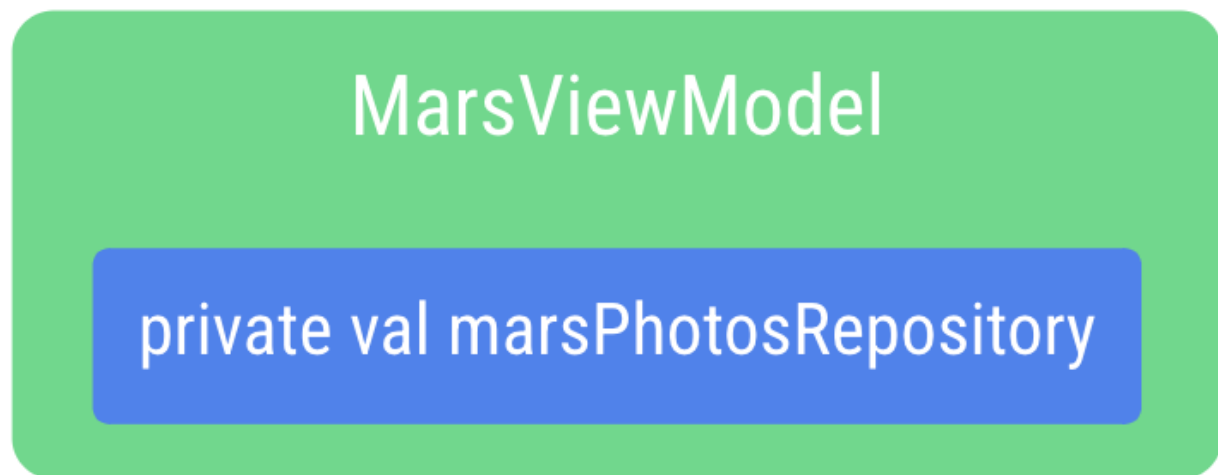


- В разделе приложения добавьте атрибут `android:name` со значением имени класса приложения «`MarsPhotosApplication`».

```
<application
    android:name=".MarsPhotosApplication"
    android:allowBackup="true"
    ...
</application>
```

Добавьте репозиторий в ViewModel

После выполнения этих шагов ViewModel может вызывать объект репозитория для получения данных о Марсе.



- Откройте файл `ui/screens/MarsViewModel.kt`.

В объявлении класса для `MarsViewModel` добавьте приватный параметр конструктора `marsPhotosRepository` типа `MarsPhotosRepository`. Значение параметра конструктора поступает из контейнера приложения, поскольку в приложении теперь используется инъекция зависимостей.

```
import com.example.marsphotos.data.MarsPhotosRepository

class MarsViewModel(private val marsPhotosRepository: MarsPhotosRepository) :
    ViewModel(){
```

- В функции `getMarsPhotos()` удалите следующую строку кода, так как теперь хранилище `marsPhotosRepository` заполняется в вызове конструктора.

```
val marsPhotosRepository = NetworkMarsPhotosRepository()
```

Поскольку фреймворк Android не позволяет передавать ViewModel значения в конструкторе при создании, мы реализуем объект ViewModelProvider.Factory, который позволяет нам обойти это ограничение.

Паттерн Factory - это паттерн создания, используемый для создания объектов. Объект `MarsViewModel.Factory` использует контейнер приложения для получения репозитория `marsPhotosRepository`, а затем передает этот репозиторий во ViewModel при создании объекта ViewModel.

Ниже функции `getMarsPhotos()` введите код для объекта-компаньона. Объект-компаньон помогает нам иметь один экземпляр объекта, который используется всеми без необходимости создавать новый экземпляр дорогостоящего объекта. Это деталь реализации, и ее разделение позволяет нам вносить изменения, не затрагивая другие части кода приложения.

`APPLICATION_KEY` является частью объекта `ViewModelProvider.AndroidViewModelFactory.Companion` и используется для поиска объекта `MarsPhotosApplication` приложения, который имеет свойство `container`, используемое для получения репозитория, используемого для инъекции зависимостей.

```
import androidx.lifecycle.ViewModelProvider
import
androidx.lifecycle.ViewModelProvider.AndroidViewModelFactory.Companion.APPLICATION
_KEY
import androidx.lifecycle.viewModelScope
import androidx.lifecycle.viewmodel.initializer
import androidx.lifecycle.viewmodel.viewModelFactory
import com.example.marsphotos.MarsPhotosApplication

companion object {
    val Factory: ViewModelProvider.Factory = viewModelFactory {
        initializer {
            val application = (this[APPLICATION_KEY] as MarsPhotosApplication)
            val marsPhotosRepository = application.container.marsPhotosRepository
            MarsViewModel(marsPhotosRepository = marsPhotosRepository)
        }
    }
}
```

- Откройте файл `theme/MarsPhotosApp.kt`, внутри функции `MarsPhotosApp()` обновите `viewModel()`, чтобы она использовала фабрику.

```
Surface(
    // ...
) {
    val marsViewModel: MarsViewModel =
        viewModel(factory = MarsViewModel.Factory)
    // ...
}
```

Эта переменная `marsViewModel` заполняется при вызове функции `viewModel()`, которой в качестве аргумента для создания `ViewModel` передается `MarsViewModel.Factory` из объекта-компаньона.

- Запустите приложение, чтобы убедиться, что оно по-прежнему ведет себя так же, как и раньше.

Поздравляем вас с рефакторингом приложения Mars Photos для использования репозитория и инъекции зависимостей! Реализация слоя данных с репозиторием позволила разделить пользовательский интерфейс и код источника данных, что соответствует лучшим практикам Android.

Благодаря использованию инъекции зависимостей стало проще тестировать `ViewModel`. Теперь ваше приложение стало более гибким, надежным и готовым к масштабированию.

После внесения этих улучшений пришло время узнать, как их тестировать. Тестирование поможет вам сохранить ожидаемое поведение кода и снизит вероятность появления ошибок при дальнейшей работе над кодом.

Настройте локальные тесты

В предыдущих разделах вы реализовали хранилище, чтобы абстрагировать прямое взаимодействие с сервисом REST API от `ViewModel`. Эта практика позволяет тестировать небольшие фрагменты кода, которые имеют ограниченное назначение. Тесты для небольших фрагментов кода с ограниченной функциональностью проще построить, реализовать и понять, чем тесты, написанные для больших фрагментов кода с множеством функциональных возможностей.

Вы также реализовали репозиторий, используя интерфейсы, наследование и инъекцию зависимостей. В следующих разделах вы узнаете, почему эти лучшие архитектурные практики упрощают тестирование. Кроме того, вы использовали корутины Kotlin для выполнения сетевых запросов. Тестирование кода, использующего корутины, требует дополнительных шагов для учета асинхронного выполнения кода. Эти шаги будут рассмотрены далее в этом уроке.

Добавление локальных зависимостей для тестирования

- Добавьте следующие зависимости в `app/build.gradle.kts`.

```
testImplementation("junit:junit:4.13.2") testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.7.1")
```

- Создайте локальный тестовый каталог

Создайте локальный тестовый каталог, щелкнув правой кнопкой мыши на каталоге `src` в представлении проекта и выбрав **New > Directory > test/java**.

- Создайте новый пакет в тестовом каталоге с именем `com.example.marsphotos`.

Создание поддельных данных и зависимостей для тестов

В этом разделе вы узнаете, как инъекция зависимостей может помочь вам в написании локальных тестов. Ранее в уроке вы создали хранилище, которое зависит от API-сервиса. Затем вы изменили модель `ViewModel`, чтобы она зависела от репозитория.

Каждый локальный тест должен проверять только одну вещь. Например, когда вы тестируете функциональность модели представления, вы не хотите тестировать функциональность хранилища или

службы API. Аналогично, тестируя хранилище, вы не хотите тестировать сервис API.

Используя интерфейсы и впоследствии используя инъекцию зависимостей для включения классов, наследующих от этих интерфейсов, вы можете **имитировать** функциональность этих зависимостей с помощью поддельных классов, созданных исключительно для целей тестирования. Инъекция поддельных классов и источников данных для тестирования позволяет тестировать код изолированно, с повторяемостью и последовательностью.

Первое, что вам понадобится, - это фальшивые данные для использования в фальшивых классах, которые вы создадите позже.

- В каталоге test создайте пакет com.example.marsphotos под названием fake.
- Создайте новый объект Kotlin в каталоге fake под названием FakeDataSource.
- В этом объекте создайте свойство, установленное на список объектов MarsPhoto. Список не должен быть длинным, но в нем должно быть не менее двух объектов.

Примечание: Для целей тестов, которые вы напишете в этом уроке, данные, хранящиеся в объектах, не обязательно должны быть репрезентативными по отношению к данным, которые возвращает API. Другими словами, вам не нужно включать действительные идентификаторы или URL-адреса.

```
object FakeDataSource {  
  
    const val idOne = "img1"  
    const val idTwo = "img2"  
    const val imgOne = "url.1"  
    const val imgTwo = "url.2"  
    val photosList = listOf(  
        MarsPhoto(  
            id = idOne,  
            imgSrc = imgOne  
        ),  
        MarsPhoto(  
            id = idTwo,  
            imgSrc = imgTwo  
        )  
    )  
}
```

Ранее в этом уроке упоминалось, что репозиторий зависит от API-сервиса. Чтобы создать тест репозитория, необходимо иметь поддельный API-сервис, который возвращает поддельные данные, которые вы только что создали. Когда этот поддельный API-сервис передается в хранилище, хранилище получает поддельные данные при вызове методов поддельного API-сервиса.

- В пакете fake создайте новый класс FakeMarsApiService. Настройте класс FakeMarsApiService так, чтобы он наследовал от интерфейса MarsApiService.

```
class FakeMarsApiService : MarsApiService {  
}
```

- Переопределите `getPhotos()`

```
override suspend fun getPhotos(): List<MarsPhoto> {  
}
```

- Верните список поддельных фотографий из метода `getPhotos()`.

```
override suspend fun getPhotos(): List<MarsPhoto> {  
    return FakeDataSource.photosList  
}
```

Помните, если вы все еще не поняли, для чего нужен этот класс, ничего страшного! Более подробно об использовании этого поддельного класса рассказывается в следующем разделе.

Тестирование репозитория

В этом разделе вы тестируете метод `getMarsPhotos()` класса `NetworkMarsPhotosRepository`. Этот раздел разъясняет использование поддельных классов и демонстрирует, как тестировать корутины.

В каталоге поддельных классов создайте новый класс `NetworkMarsRepositoryTest`.

- Создайте новый метод в только что созданном классе под названием `networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList()` и аннотируйте его с помощью `@Test`.

```
@Test  
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList(){  
}
```

Чтобы протестировать хранилище, вам понадобится экземпляр `NetworkMarsPhotosRepository`. Напомним, что этот класс зависит от интерфейса `MarsApiService`. Именно здесь вы используете поддельную службу API из предыдущего раздела.

Создайте экземпляр `NetworkMarsPhotosRepository` и передайте `FakeMarsApiService` в качестве параметра `marsApiService`.

```
@Test  
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList(){  
    val repository = NetworkMarsPhotosRepository(  
        marsApiService = FakeMarsApiService()  
    )  
}
```

Передавая фальшивый API-сервис, любые обращения к свойству `marsApiService` в репозитории приводят к вызову `FakeMarsApiService`. Передавая поддельные классы для зависимостей, вы можете контролировать, что именно возвращает зависимость. Такой подход гарантирует, что тестируемый код не будет зависеть от непроверенного кода или API, которые могут измениться или иметь непредвиденные проблемы. Такие ситуации могут привести к тому, что ваш тест окажется неудачным, даже если в написанном вами коде все в порядке. Подделки помогают создать более последовательное тестовое окружение, уменьшить количество ошибок в тестах и упростить лаконичные тесты, проверяющие одну функциональность.

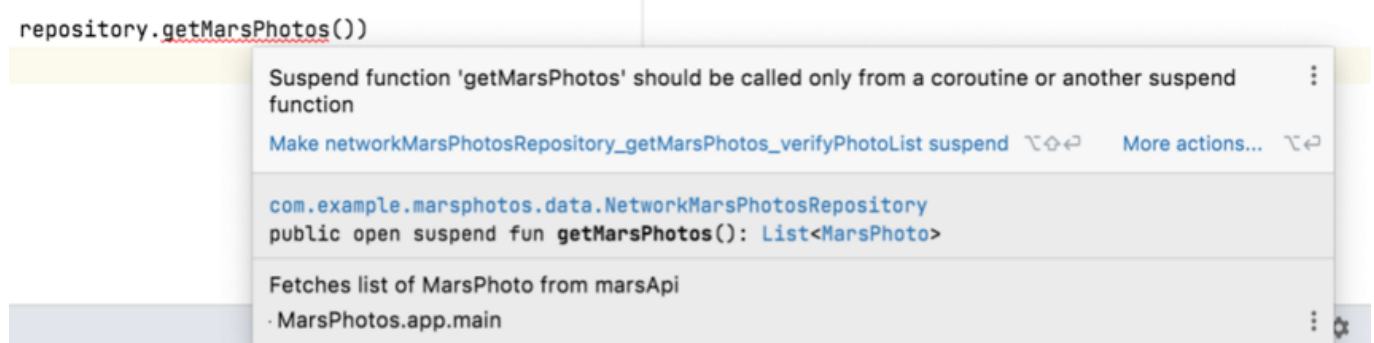
Убедитесь, что данные, возвращаемые методом `getMarsPhotos()`, равны списку `FakeDataSource.photosList`.

```
@Test
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList(){
    val repository = NetworkMarsPhotosRepository(
        marsApiService = FakeMarsApiService()
    )assertEquals(FakeDataSource.photosList, repository.getMarsPhotos())
}
```

Обратите внимание, что в вашей IDE вызов метода `getMarsPhotos()` подчеркнут красным цветом.

```
@Test
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList() {
    val repository = NetworkMarsPhotosRepository(
        marsApiService = FakeMarsApiService()
    )
    assertEquals(FakeDataSource.photosList, repository.getMarsPhotos())
}
```

Если вы наведете курсор на метод, то увидите всплывающую подсказку, указывающую, что «Приостановленная функция „getMarsPhotos“ должна вызываться только из корутины или другой приостановленной функции:».



В файле `data/MarsPhotosRepository.kt`, рассматривая реализацию `getMarsPhotos()` в `NetworkMarsPhotosRepository`, вы видите, что функция `getMarsPhotos()` является функцией

приостановки.

```
class NetworkMarsPhotosRepository(
    private val marsApiService: MarsApiService
) : MarsPhotosRepository {
    /** Fetches list of MarsPhoto from marsApi*/
    override suspend fun getMarsPhotos(): List<MarsPhoto> =
        marsApiService.getPhotos()
}
```

Помните, когда вы вызывали эту функцию из модели `MarsViewModel`, вы вызывали этот метод из корутины, вызывая его из лямбды, переданной в `viewModelScope.launch()`. Вы также должны вызывать приостановленные функции, такие как `getMarsPhotos()`, из корутины в тесте. Однако подход здесь другой. В следующем разделе мы рассмотрим, как решить эту проблему.

Тестовые корутины В этом разделе вы измените тест `networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList()` таким образом, чтобы тело метода теста запускалось из корутины.

Измените в файле `NetworkMarsRepositoryTest.kt` функцию `networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList()` так, чтобы она представляла собой выражение.

```
@Test
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList() =
```

Установите выражение, равное функции `runTest()`. Этот метод ожидает лямбду.

```
...
import kotlinx.coroutines.test.runTest
...

@Test
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList() =
    runTest {}
```

Библиотека тестов `coroutine` предоставляет функцию `runTest()`. Эта функция принимает метод, который вы передали в лямбде, и запускает его из `TestScope`, который наследуется от `CoroutineScope`.

Примечание: для справки, когда вы вызываете хранилище в созданной вами `ViewModel`, вы вызываете `getMarsPhotos()` с помощью `viewModelScope`, который в конечном итоге является `CoroutineScope`.

Переместите содержимое тестовой функции в лямбда-функцию.

```
@Test
fun networkMarsPhotosRepository_getMarsPhotos_verifyPhotoList() =
    runTest {
        val repository = NetworkMarsPhotosRepository(
            marsApiService = FakeMarsApiService()
        )
        assertEquals(FakeDataSource.photosList, repository.getMarsPhotos())
    }
```

Обратите внимание, что красная линия под `getMarsPhotos()` теперь исчезла. Если вы запустите этот тест, он пройдет!

10. Напишем тест для ViewModel

В этом разделе вы напишете тест для функции `getMarsPhotos()` из модели `MarsViewModel`. Модель `MarsViewModel` зависит от хранилища `MarsPhotosRepository`. Поэтому, чтобы написать этот тест, вам нужно создать поддельное хранилище `MarsPhotosRepository`. Кроме того, для корутинов необходимо учесть некоторые дополнительные шаги, помимо использования метода `runTest()`.

Создание поддельного репозитория Цель этого шага - создать поддельный класс, который наследует интерфейс `MarsPhotosRepository` и переопределяет функцию `getMarsPhotos()`, чтобы вернуть поддельные данные. Этот подход похож на тот, который вы использовали при создании поддельного API-сервиса, с той лишь разницей, что этот класс расширяет интерфейс `MarsPhotosRepository`, а не `MarsApiService`.

Создайте новый класс в каталоге подделок под названием `FakeNetworkMarsPhotosRepository`. Расширьте этот класс интерфейсом `MarsPhotosRepository`.

```
class FakeNetworkMarsPhotosRepository : MarsPhotosRepository{
}
```

Переопределите функцию `getMarsPhotos()`.

```
class FakeNetworkMarsPhotosRepository : MarsPhotosRepository{
    override suspend fun getMarsPhotos(): List<MarsPhoto> {
    }
}
```

Возвращает список фотографий `FakeDataSource.photosList` из функции `getMarsPhotos()`.

```
class FakeNetworkMarsPhotosRepository : MarsPhotosRepository{
    override suspend fun getMarsPhotos(): List<MarsPhoto> {
        return FakeDataSource.photosList
    }
}
```

```
}
}
```

Напишите тест ViewModel Создайте новый класс с именем MarsViewModelTest. Создайте функцию marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess() и аннотируйте ее с помощью @Test.

```
@Test
fun marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess()
```

Сделайте эту функцию выражением, установленным на результат метода runTest(), чтобы тест выполнялся из корутины, как и тест репозитория в предыдущем разделе.

```
@Test
fun marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess() =
    runTest{
    }
```

В теле лямбды runTest() создайте экземпляр MarsViewModel и передайте ему экземпляр созданного вами фальшивого репозитория.

```
@Test
fun marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess() =
    runTest{
        val marsViewModel = MarsViewModel(
            marsPhotosRepository = FakeNetworkMarsPhotosRepository()
        )
    }
```

Убедитесь, что marsUiState вашего экземпляра ViewModel совпадает с результатом успешного вызова MarsPhotosRepository.getMarsPhotos().

Примечание: Вам не нужно напрямую вызывать MarsViewModel.getMarsPhotos(), чтобы инициировать вызов MarsPhotosRepository.getMarsPhotos(). MarsViewModel.getMarsPhotos() вызывается при инициализации ViewModel.

```
@Test
fun marsViewModel_getMarsPhotos_verifyMarsUiStateSuccess() =
    runTest {
        val marsViewModel = MarsViewModel(
            marsPhotosRepository = FakeNetworkMarsPhotosRepository()
        )
        assertEquals(
            MarsUiState.Success("${FakeDataSource.photosList.size} Mars "
+

```

```
        "photos retrieved"),  
        marsViewModel.marsUiState  
    )  
}
```

Если вы попытаетесь запустить этот тест как есть, он завершится неудачей. Ошибка будет выглядеть примерно так, как показано в следующем примере:

```
Exception in thread "Test worker @coroutine#1" java.lang.IllegalStateException:  
Module with the Main dispatcher had failed to initialize. For tests  
Dispatchers.setMain from kotlinx-coroutines-test module can be used
```

Напомним, что `MarsViewModel` вызывает хранилище с помощью `viewModelScope.launch()`. Эта инструкция запускает новую корутину под диспетчером корутин по умолчанию, который называется диспетчером `Main`. Диспетчер `Main` оборачивает поток пользовательского интерфейса `Android`. Причина предыдущей ошибки заключается в том, что поток пользовательского интерфейса `Android` недоступен в модульном тесте. Модульные тесты выполняются на вашей рабочей станции, а не на устройстве или эмуляторе `Android`. Если код локального модульного теста ссылается на диспетчер `Main`, то при запуске модульных тестов будет выброшено исключение (как показано выше). Чтобы решить эту проблему, необходимо явно определить диспетчер по умолчанию при запуске модульных тестов. Перейдите к следующему разделу, чтобы узнать, как это сделать.

Создание тестового диспетчера Поскольку диспетчер `Main` доступен только в контексте пользовательского интерфейса, необходимо заменить его диспетчером, удобным для юнит-тестов. Библиотека `Kotlin Coroutines` предоставляет для этой цели диспетчер корутин под названием `TestDispatcher`. `TestDispatcher` необходимо использовать вместо диспетчера `Main` для любого юнит-теста, в котором создается новая корутина, как в случае с функцией `getMarsPhotos()` из модели представления.

Чтобы заменить диспетчер `Main` на `TestDispatcher` во всех случаях, используйте функцию `Dispatchers.setMain()`. Вы можете использовать функцию `Dispatchers.resetMain()`, чтобы вернуть диспетчер потока обратно к диспетчеру `Main`. Чтобы не дублировать код, заменяющий диспетчер `Main`, в каждом тесте, можно извлечь его в тестовое правило `JUnit`. `TestRule` предоставляет возможность контролировать окружение, в котором выполняется тест. `TestRule` может добавлять дополнительные проверки, выполнять необходимую настройку или очистку тестов, а также наблюдать за выполнением теста, чтобы сообщить о нем в другом месте. Они могут быть легко разделены между тестовыми классами.

Создайте специальный класс для написания `TestRule`, чтобы заменить диспетчер `Main`. Чтобы реализовать пользовательское `TestRule`, выполните следующие шаги:

Создайте новый пакет в каталоге `test` под названием `rules`. В каталоге `rules` создайте новый класс `TestDispatcherRule`. Расширьте `TestDispatcherRule` классом `TestWatcher`. Класс `TestWatcher` позволяет выполнять действия на различных этапах выполнения теста.

```
class TestDispatcherRule(): TestWatcher(){

}
```

Создайте параметр конструктора TestDispatcher для правила TestDispatcherRule. Этот параметр позволяет использовать различные диспетчеры, например StandardTestDispatcher. Этот параметр конструктора должен иметь значение по умолчанию, установленное на экземпляр объекта UnconfinedTestDispatcher. Класс UnconfinedTestDispatcher наследуется от класса TestDispatcher и определяет, что задания не должны выполняться в каком-либо определенном порядке. Такая схема выполнения хороша для простых тестов, поскольку корутины обрабатываются автоматически. В отличие от UnconfinedTestDispatcher, класс StandardTestDispatcher позволяет полностью контролировать выполнение корутин. Этот способ предпочтителен для сложных тестов, требующих ручного подхода, но для тестов в этом коделабе он не нужен.

```
class TestDispatcherRule(
    val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(),
) : TestWatcher() {

}
```

```
class TestDispatcherRule( val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(), ) : TestWatcher() {

}
```

```
class TestDispatcherRule(
    val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(),
) : TestWatcher() {
    override fun starting(description: Description) {

    }
}
```

Добавьте вызов Dispatchers.setMain(), передав в качестве аргумента testDispatcher.

```
class TestDispatcherRule(
    val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(),
) : TestWatcher() {
    override fun starting(description: Description) {
        Dispatchers.setMain(testDispatcher)
    }
}
```

После завершения выполнения теста сбросьте диспетчер Main, переопределив метод finished(). Вызовите функцию Dispatchers.resetMain().


```
class TestDispatcherRule(  
    val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(),  
) : TestWatcher() {  
    override fun starting(description: Description) {  
        Dispatchers.setMain(testDispatcher)  
    }  
  
    override fun finished(description: Description) {  
        Dispatchers.resetMain()  
    }  
}
```

Правило TestDispatcherRule готово к повторному использованию.

Откройте файл MarsViewModelTest.kt. В классе MarsViewModelTest создайте класс TestDispatcherRule и присвойте его свойству testDispatcher, доступному только для чтения.

```
class MarsViewModelTest {  
  
    val testDispatcher = TestDispatcherRule()  
    ...  
}
```

Чтобы применить это правило к своим тестам, добавьте аннотацию @get:Rule к свойству testDispatcher.

```
class MarsViewModelTest {  
    @get:Rule  
    val testDispatcher = TestDispatcherRule()  
    ...  
}
```

Повторно запустите тест. Убедитесь, что на этот раз он пройден.

Заключение

Поздравляем вас с завершением этого урока и рефакторингом приложения **Mars Photos** для реализации паттерна репозитория и инъекции зависимостей!

Теперь код приложения соответствует лучшим практикам Android для слоя данных, а это значит, что он стал более гибким, надежным и легко масштабируемым.

Эти изменения также помогли сделать приложение более легко тестируемым. Это очень важно, так как код может продолжать развиваться и при этом быть уверенным, что он по-прежнему ведет себя так, как ожидается.